

ThriftyAI Platform — QA Testing Report

Dashboard Functional Testing & Issue Log — platform.thriftyai.in

Report Date: 10 July 2026

Environment: Production (platform.thriftyai.in)

Total Issues Logged: 32

19

NOT WORKING

9

PARTIAL / UX ISSUE

4

SUGGESTIONS

3

MODULES NOT TESTED

Detailed Findings

#	FEATURE	TEST SCENARIO / FLOW	STATUS	ISSUES FOUND	CHANGES REQUIRED	REMARKS / RECOMMENDATIONS
A. GENERAL / PLATFORM-WIDE						
1	Feature Agent API	Call <code>POST /api/feature-agent/parse</code> to parse a feature prompt	Not Working	API returns 500 InternalServerError. Root cause: 401 “Incorrect API key provided” from OpenAI — the configured OpenAI key (sk-proj-...OAAA) is invalid/expired. Full stack trace is exposed in the API response.	Replace/renew the OpenAI API key in backend environment config. Remove stack traces & internal file paths from production error responses.	Exposing stack traces and partial API keys in responses is a security risk. Add centralized error sanitization for production.
2	Navigation / Page Loader	Navigate between any dashboard menu items	Partial	The full-screen Thrifty loader appears on every menu click / route change, which is frustrating for client users.	Load shared layout once; use skeleton loaders or inline spinners for content area only. Cache previously fetched data (e.g. SWR / React Query).	Full-page branded loaders should only appear on initial app load, not on every internal navigation.
3	Home — “Your Own Avatar”	Click “Your Own Avatar” card on home page	Partial	Card is labelled “Coming Soon” but the link still redirects to the feature page — contradictory behaviour.	Either disable the link while it is “Coming Soon”, or remove the “Coming Soon” label if the feature is live.	Inconsistent state confuses users about feature availability.
24	Session / Idle Timeout	Leave the system idle, return, and perform any action	Partial	First API request after idle period logs the user out because the session has already expired server-side.	Implement sliding session expiry (reset timeout on user activity) and/or refresh-token flow with token stored in localStorage; silently refresh on 401 before forcing logout.	Poor UX — users lose work context. Warn the user before expiry (“session about to expire” modal) as an additional improvement.
B. VIDEOS MODULE						

B. VIDEOS MODULE

#	FEATURE	TEST SCENARIO / FLOW	STATUS	ISSUES FOUND	CHANGES REQUIRED	REMARKS / RECOMMENDATIONS
4	Video Creation — Progress	Start video creation on <code>/dashboard/videos</code> , navigate away, then return to the page	Not Working	On returning, progress shows an inflated value (e.g. 48%) and then drops to the real value (e.g. 37%) after a few seconds — stale/optimistic progress state is rendered first.	Fetch the real job progress from the API on component mount before rendering; do not render cached/simulated progress. Use polling or WebSocket for live progress.	Progress must be single-source-of-truth from backend; never let a client-side fake counter run ahead of actual job progress.
5	Videos — Filter	Use filter button on <code>/dashboard/videos?job_id=...</code>	Not Working	Filter button does nothing; list is not filtered by the selected criteria / <code>job_id</code> .	Wire the filter button to the list query; apply <code>job_id</code> and other filter params to the API request.	Verify URL query params are read on page load so deep-linked filters also work.
6	Video Details API (in-progress job)	Call <code>GET /api/video/:id</code> while the video job is still processing	Not Working	API returns 500 InternalServerError — “ Job is not completed yet ” with full stack trace. An in-progress job is an expected state, not a server error.	Return a proper status (e.g. 200 with <code>status: "processing"</code> and progress, or 202/409) instead of throwing. Frontend should render a processing state.	500s pollute error monitoring and mislead the client. Same fix as #1: strip stack traces from production responses.
7	Video Share Modal	Open a completed video → Share → click GIF and Embed buttons	Not Working	GIF and Embed buttons in the share modal do not respond / perform no action.	Implement GIF export and Embed-code copy, or hide the buttons until implemented.	Dead buttons in a share flow directly reduce trust; remove or finish them.
8	Video Share — Password Protection	Enable “Require password” toggle when sharing	Not Working	Toggle exists but has no effect — no password attachment/entry feature is present anywhere; toggle is not tied to any user/asset.	Implement password-protected share links end-to-end (set password, enforce on public view page), or remove the toggle.	A security-looking control that silently does nothing is worse than not having it — users may believe content is protected.
C. FILE UPLOADS / KNOWLEDGE BASE						
9	KB Files — Thumbnail Upload	Upload thumbnail attachment via dashboard → <code>POST /api/knowledge-bases/:id/files</code>	Not Working	nginx returns raw 413 Request Entity Too Large HTML page; UI shows only a generic “an error occurred” message.	(1) Validate file size on client before upload and show a clear toaster: “File exceeds max size of X MB”. (2) Align <code>nginx client_max_body_size</code> with the intended app limit. (3) Map 413 responses to a friendly error message.	See consolidated 413 note below — the file being uploaded is larger than the nginx body-size limit; the failure is at the proxy, so the app never sees the request.
20	KB Files — Upload	Upload files to knowledge base <code>POST /api/knowledge-bases/:id/files</code>	Not Working	Files are not getting uploaded at all for KB <code>caa01895-...</code>	Debug upload pipeline (proxy limit, multipart handling, storage write); surface the specific failure reason to the UI.	Likely related to the same 413 / body-size limit issue; verify with files under the limit to isolate.
23	KB — AI Processing / Retry	Preview knowledge base; retry AI processing via <code>.../files/:fileId/retry-ai</code>	Not Working	<code>ValidationError: “KB does not have a Pinecone assistant configured”</code> appears during AI processing when previewing the KB.	Auto-provision the Pinecone assistant when a KB is created, or block AI-processing actions with a clear setup message until it is configured.	Users cannot configure Pinecone themselves — this must be handled by backend provisioning, not surfaced as a user-facing validation error.
D. AVATARS MODULE (CREATE / EDIT)						
10	Avatar Create — Experience Builder	Click “Learn more” in Experience Builder step (<code>/dashboard/avatars/create</code>)	Not Working	“Learn more” button is a mock — performs no action.	Link it to real documentation/help content, or remove it.	—
11	Avatar Create — Experience Type	Select “Guided mode”, go to next step, come back, and change experience type	Not Working	On returning to the step, changing the experience type/level does not update — the previous selection stays applied.	Fix wizard state management so revisited steps re-bind to current form state and edits propagate.	Applies to the whole wizard — every step must remain editable when navigated back to.

#	FEATURE	TEST SCENARIO / FLOW	STATUS	ISSUES FOUND	CHANGES REQUIRED	REMARKS / RECOMMENDATIONS
12	Avatar Create — Step Update	Modify any value after navigating back to a previous wizard step	Not Working	Updates made after coming back to a step are not saved/applied.	Same fix as #11 — wizard steps must read from and write to a single form state store.	Add step-level validation on “Next” so state is committed consistently.
13	Avatar Create — Preview	Click “Preview” button on <code>/dashboard/avatars/create</code>	Not Working	Preview button does not respond.	Implement avatar preview or disable the button with a tooltip until ready.	—
14	Avatar — Edit Experience Tasks	In Edit Experience, use Delete and Copy buttons on tasks	Not Working	Both Delete and Copy buttons on experience tasks do nothing.	Wire delete/copy actions to task list state and persist changes.	—
15	Avatar Create — Voice Selection	Open “Select voice” dropdown → use filter button	Not Working	Filter button inside the voice dropdown does not work.	Implement voice filtering (by language/gender/style) or remove the filter control.	—
16	Avatar — Test Experience	Open avatar in edit mode (<code>?edit=b355f458-...</code>) → click “Test Experience”	Not Working	Test Experience always fails with “ Session expired ”.	Fix the auth/session token passed to the test-experience session (likely wrong/expired token or missing refresh).	May be related to issue #24 (session expiry handling).
17	Avatar — Edit	Edit an existing avatar via <code>/dashboard/avatars/create?edit=...</code>	Not Working	Edit button/flow does not work properly — avatar cannot be reliably edited.	Ensure edit mode loads existing avatar data into the wizard and the save path performs an update (not create).	Overlaps with #11/#12 wizard state issues.
18	Custom Avatar — Create	Create custom avatar → <code>POST /api/custom-avatars</code>	Not Working	nginx returns raw 413 Request Entity Too Large ; custom avatar is never created.	Same as #9: client-side file-size validation with clear toaster + align <code>nginx client_max_body_size</code> ; handle 413 gracefully.	See consolidated 413 note below — the uploaded image/video exceeds the proxy body-size limit.
19	Custom Avatar — Delete	Delete a custom avatar from the list	Partial	Success toaster appears but avatar remains in the list. Deleting again returns “Avatar not found or not owned by organization” — so the delete actually succeeded but the list never refreshed.	Refresh/invalidate the avatar list (or optimistically remove the item) after successful delete.	Classic stale-cache bug; second error message confirms backend delete works.
E. PRESENTATIONS (PPT) MODULE						
21	PPT — Themes API	Open presentation creation form → <code>GET /api/ppt/themes</code>	Not Working	Themes API errors during form fetch; theme list never loads.	Fix the themes endpoint; add error/retry state in the form instead of silently failing.	Root cause of #22.
22	PPT — Create Presentation	Create a presentation	Not Working	Creation fails because no theme is selected — direct cascade of #21 (themes never loaded).	Fix #21; additionally provide a default theme fallback so creation is not hard-blocked by a theme-list failure.	Validate the form with a helpful message (“please select a theme”) rather than a raw API error.
F. CAMPAIGNS MODULE						
25	Campaign Details — Click Tracking	View campaign details after emails are opened/clicked	Suggestion	Opened time is shown, but clicked information is not available. Tracking pixels only detect opens, not clicks.	Implement tracking links (redirect links) that log the click and then redirect to the destination URL.	Standard approach: wrap all campaign URLs in a redirect endpoint per recipient for accurate click analytics.
26	Campaign Details — Video Campaigns	Open a sent item in a video campaign (<code>/dashboard/campaigns/:id</code>)	Suggestion	Options like Costs, Log, Analysis, Transcripts, Messages are shown even for video campaigns where they serve no purpose — they are only meaningful for interactive campaigns.	Conditionally render these panels based on campaign type: hide for video campaigns, keep for interactive campaigns.	Reduces noise and confusion in the campaign detail view.

#	FEATURE	TEST SCENARIO / FLOW	STATUS	ISSUES FOUND	CHANGES REQUIRED	REMARKS / RECOMMENDATIONS
G. CONVERSATIONS / ANALYTICS / SESSION LOGS						
27	Conversations Analytics — Filters	Use filters button on /dashboard/conversations/analytics	Not Working	Filters button does not work.	Wire the filter controls to the analytics query.	—
28	Session Logs — Search	Search session logs	Partial	Search only matches <code>room_id</code> ; other fields are not searchable.	Extend search to cover <code>room_id</code> plus other relevant fields (user, agent, status, date, etc.).	Consider a field-scoped search or a global “search all columns” behaviour.
29	Conversations Analytics — Row Checkboxes	Select rows using per-row checkboxes	Partial	Checkboxes exist on every row but no bulk operation (bulk delete, export selected, etc.) is available — selection has no purpose.	Either add bulk actions (delete, export) for selected rows or remove the checkboxes.	UI affordances must map to real actions.
30	Session Logs — Pagination	Change “records per page” on the Session Logs table	Not Working	Records-per-page selector has no effect on the table.	Bind page-size selection to the table query (limit/offset) and refetch.	—
H. WORKSPACES MODULE						
31	Workspace — Create	Add a new workspace → <code>POST /api/workspaces</code>	Partial	API rejects with validation error: “ <i>Slug must contain only lowercase letters, numbers, hyphens, and underscores</i> ” — but the raw JSON error is not surfaced to the user as a readable message.	(1) Show the validation message in a proper toaster. (2) Better: auto-generate a valid slug from the workspace name (lowercase, spaces → hyphens) so users never hit this.	Client-side validation should mirror server rules to prevent avoidable round-trips.
32	Workspace — Delete	Delete a workspace	Not Working	Shows “Workspace deletion feature coming soon!” — deletion is not implemented.	Implement workspace deletion (with confirmation dialog and cascade/archival rules for contained data).	Deletion is a basic requirement for any workspace system; prioritize.

Consolidated Note — 413 “Request Entity Too Large” Errors (Issues #9, #18, #20)

Root Cause

These errors occur because the **uploaded file size exceeds the request body-size limit configured on the nginx reverse proxy** (`client_max_body_size`, `nginx/1.24.0`). The request is rejected at the proxy layer before it ever reaches the application, so nginx returns its raw HTML 413 page and the frontend only shows a generic “an error occurred” message.

Required Fixes

- **Client-side validation:** check the file size before uploading and show a proper toaster, e.g. “*File is too large. Maximum allowed size is X MB.*” — never let the user wait through a doomed upload.
- **Server/proxy alignment:** set nginx `client_max_body_size` to match the intended application upload limit (and the backend body-parser limit) so the limits are consistent everywhere.
- **Graceful 413 handling:** if a 413 still slips through, the frontend should detect the status code and show the same friendly size-limit message instead of a generic error.
- **Show the limit up-front:** display the maximum allowed file size next to every upload control.

Modules Not Yet Tested / Pending

Pending Coverage

The following modules are **left for testing** and are not covered in this report. They must be tested in the next QA cycle:

- **Subscription** — plans, billing, upgrade/downgrade, payment flows.
- **Referral** — referral link generation, tracking, and rewards.

Overall Recommendations

- **Error hygiene:** multiple APIs (#1, #6) return HTTP 500 with full stack traces, internal file paths, and partial secrets for expected conditions. Sanitize all production error responses and use correct status codes.
- **Dead/mock UI:** several buttons are non-functional or mocked (#7, #8, #10, #13, #14, #15, #27, #29). Either implement them or remove/disable them — dead controls erode user trust.
- **Wizard state management:** the avatar creation wizard (#11, #12, #17) needs a single source of truth for form state so back-navigation and edits work reliably.
- **Upload limits:** resolve the 413 cluster (#9, #18, #20) with client-side size validation, visible limits, and aligned proxy/backend configuration.
- **Session handling:** implement sliding expiry / silent token refresh (#24) — also likely fixes the “Test Experience” session-expired issue (#16).
- **List freshness:** invalidate/refetch lists after mutations (#19) so the UI always reflects backend state.